Implementasi dan Analisis *Alpha-Beta Pruning* serta *Monte Carlo Tree Search* pada Permainan Othello

Kelompok B15

Anggota Kelompok:

1. Ahmad Fateh Eydil Faiq 5025251234
2. Irsyad Ansyari Hakim 5025251242
3. Hamizan Rifqi Afandi 5025251246
4. Darryl Ardan Wicaksono 5025251253

BAB I — Pendahuluan

1.1 Latar Belakang

Adversarial Search merupakan pendekatan pencarian keputusan dalam lingkungan kompetitif. Suatu tindakan tidak hanya dinilai dari manfaat langsungnya, tetapi juga dari kemungkinan tanggapan lawan. Pada permainan dua pemain, keputusan dapat dimodelkan sebagai penelusuran pohon permainan yang memuat posisi papan, langkah legal, dan pergantian giliran.

Othello dipilih karena mempunyai aturan lokal yang sederhana, tetapi konsekuensi keputusan dapat menjangkau banyak giliran. Satu langkah dapat membalik sejumlah bidak sekaligus, mengubah mobilitas lawan, dan membuka atau menutup akses ke sudut. Oleh karena itu, banyaknya bidak pada suatu giliran belum cukup untuk menilai kualitas posisi secara keseluruhan.

Proyek ini membandingkan dua pendekatan. Alpha-Beta Pruning melakukan pencarian Minimax hingga kedalaman tertentu dengan fungsi evaluasi posisi. MCTS mengembangkan pohon secara bertahap berdasarkan hasil simulasi permainan. Implementasi pertama deterministik untuk posisi dan konfigurasi yang sama, sedangkan implementasi kedua menggunakan pengacakan yang dapat direproduksi melalui seed.

Perbandingan keduanya penting untuk memahami hubungan antara biaya pencarian dan kualitas keputusan. Depth dan iterations merupakan pengendali pekerjaan yang berbeda, sehingga kesamaan nilai parameternya tidak berarti kesamaan biaya komputasi. Evaluasi harus mempertimbangkan waktu, memori, statistik internal, dan hasil pertandingan secara bersamaan.

1.2 Rumusan Masalah

1. Bagaimana mengimplementasikan Alpha-Beta Pruning dan MCTS pada permainan Othello dengan kontrak data dan aturan yang konsisten?

2. Bagaimana perbandingan kompleksitas waktu dan ruang kedua algoritma secara teoretis serta berdasarkan data empiris yang tersedia?
3. Bagaimana depth dan iterations mengendalikan proses pencarian, dan sejauh mana pengaruhnya terhadap performa dapat dibuktikan melalui eksperimen?
4. Bagaimana kemampuan bermain kedua algoritma ketika warna dipertukarkan, serta bagaimana karakteristik posisi opening, midgame, dan endgame digunakan dalam pengujian?

1.3 Tujuan

Tujuan proyek adalah membangun permainan Othello yang dapat dimainkan melalui GUI, menyediakan dua algoritma AI dengan antarmuka pemanggilan yang konsisten, dan memisahkan aturan permainan dari antarmuka pengguna. Proyek juga bertujuan menyediakan pengukuran waktu dan memori yang dapat diulang, pertandingan otomatis dengan pertukaran warna, serta skenario yang dapat diverifikasi melalui riwayat langkah legal.

Analisis dilakukan untuk menjelaskan mekanisme pencarian dan menghubungkannya dengan hasil yang tersedia. Kesimpulan dibatasi oleh cakupan dataset agar hasil dua pertandingan atau satu konfigurasi benchmark tidak digeneralisasi menjadi keunggulan universal suatu algoritma.

1.4 Batasan Masalah

1. Permainan menggunakan papan standar 8×8, dua pemain, informasi sempurna, dan hitam bergerak terlebih dahulu.
2. Algoritma yang diteliti adalah Minimax dengan Alpha-Beta Pruning dan MCTS dengan UCT.
3. Alpha-Beta menggunakan heuristik tetap `25C + 5M + D`; nilai terminal dipisahkan dari evaluasi nonterminal.
4. MCTS menggunakan konstanta eksplorasi `sqrt(2)`, rollout acak uniform, dan reward menang/seri/kalah sebesar `1.0/0.5/0.0`.
5. Program menggunakan Python dan Streamlit. Benchmark dan tournament dapat berjalan tanpa GUI.
6. Metrik memori adalah peak alokasi Python yang dilacak `tracemalloc`, bukan total resident memory proses.
7. Dataset benchmark yang dianalisis hanya mencakup MCTS, midgame, 500 iterasi, dan 30 pengulangan. Dataset pertandingan berisi satu pasangan warna pada depth 4 dan 500 iterasi.
8. Belum dilakukan eksperimen batas waktu berpikir yang sama, penyapuan parameter, maupun pengujian kemampuan bermain dari beragam posisi awal.
9. Tidak digunakan transposition table, pembelajaran bobot heuristik, atau rollout berbasis pengetahuan posisi.

BAB II — Landasan Teori

2.1 Adversarial Search

Dalam model proyek, state terdiri dari papan dan pemain yang mendapat giliran. Action merupakan koordinat penempatan bidak yang legal. Transition menghasilkan papan berikutnya setelah penempatan dan pembalikan bidak. Terminal state terjadi ketika permainan tidak dapat dilanjutkan oleh kedua pemain. Utility menyatakan hasil akhir dari perspektif pemain tertentu.

Game tree dibentuk dengan menjadikan state awal sebagai root dan state setelah action sebagai child. Node MAX memilih nilai terbesar dari perspektif pemain root, sedangkan node MIN memilih nilai terkecil. Penetapan MAX atau MIN harus mengikuti pemain yang benar-benar mendapat giliran. Pada Othello, pass dapat menyebabkan pemain yang sama bermain kembali, sehingga label MAX dan MIN tidak selalu berganti hanya berdasarkan kedalaman node.

Pada proyek ini, hasil akhir didefinisikan sebagai menang, kalah, atau seri berdasarkan jumlah bidak. Sebelum terminal, kualitas posisi diperkirakan melalui heuristik atau statistik simulasi sesuai algoritma yang digunakan.

2.2 Permainan Othello

Pada awal permainan terdapat empat bidak pada pusat papan. Langkah dilakukan pada kotak kosong yang menjepit sedikitnya satu bidak lawan dalam garis horizontal, vertikal, atau diagonal. Semua bidak lawan yang terjepit akibat langkah tersebut dibalik. Pemain yang tidak mempunyai langkah wajib pass; permainan berakhir ketika kedua pemain tidak dapat bergerak, dan jumlah bidak menentukan pemenang. Aturan ini mengikuti aturan Othello yang dipublikasikan World Othello Federation. Aturan resmi Othello.

Representasi dalam proyek menggunakan `EMPTY = 0`, `BLACK = 1`, dan `WHITE = -1`. Koordinat `(row, col)` menggunakan indeks 0–7, dengan baris nol di atas dan kolom nol di kiri. Penggunaan bilangan bertanda membuat lawan pemain dapat ditentukan melalui `-player`.

2.3 Minimax dan Alpha-Beta Pruning

Minimax memilih tindakan dengan mengasumsikan bahwa lawan juga memilih tanggapan terbaik. Alpha-Beta mempertahankan hasil Minimax pada pohon dan evaluasi yang sama sambil menghindari pemeriksaan cabang yang tidak lagi dapat memengaruhi keputusan. Alpha adalah batas bawah nilai yang dapat dipertahankan MAX, sedangkan beta adalah batas atas yang dapat dipertahankan MIN. Cutoff dilakukan ketika `alpha >= beta`. Urutan pemeriksaan langkah memengaruhi besarnya penghematan pencarian.

Dalam implementasi proyek, pencarian dihentikan ketika depth tersisa nol atau posisi terminal. Leaf nonterminal menggunakan heuristik; leaf terminal menggunakan hasil

kemenangan. Langkah diperiksa dalam urutan koordinat sehingga hasil deterministik dan pemilihan pada nilai seri mengikuti koordinat terkecil.

2.4 Monte Carlo Tree Search

MCTS menggabungkan pengembangan pohon dengan perkiraan nilai melalui simulasi. Empat tahap dasarnya adalah selection untuk memilih jalur, expansion untuk menambahkan child, simulation untuk memperoleh hasil permainan, dan backpropagation untuk memperbarui statistik jalur. Browne dkk., A Survey of Monte Carlo Tree Search Methods.

Selection menggunakan Upper Confidence Bound applied to Trees atau UCT:

$$UCT_i = \overline{X_i} + c\sqrt{\frac{\ln N}{n_i}}.$$

Dalam proyek, child menyimpan reward dari perspektif pemain yang memilih tindakan pada parent. Karena itu, selection pada giliran lawan tetap memaksimalkan kepentingan lawan, bukan reward root secara membuta. Nilai `c` ditetapkan `sqrt(2)` dan keputusan akhir memakai visit count terbesar, dengan koordinat terkecil sebagai pemecah seri.

2.5 Kompleksitas Algoritma

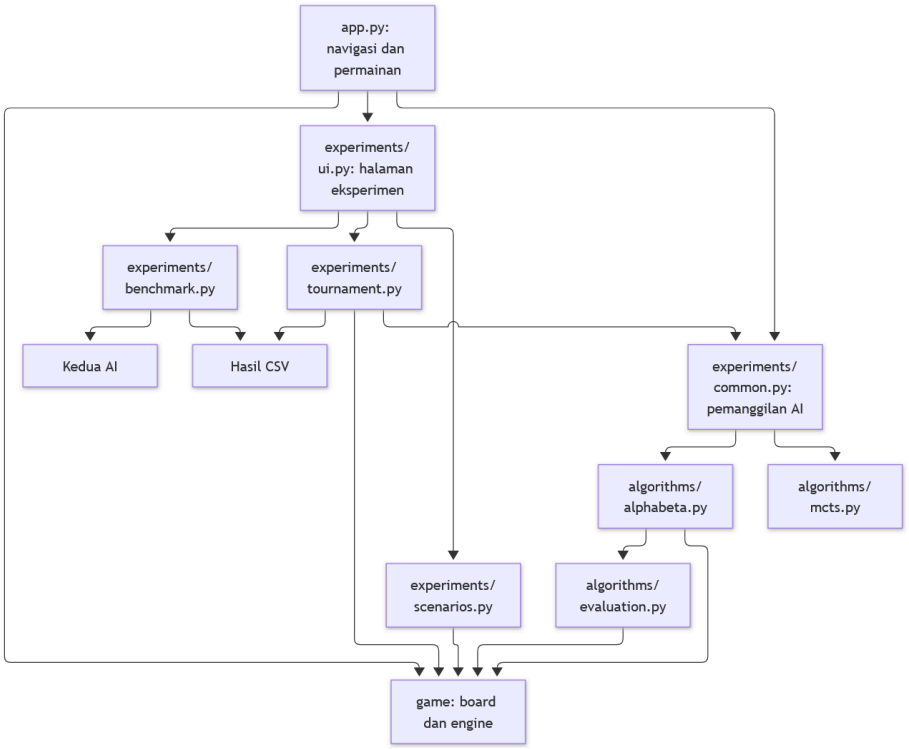
Konsep time complexity, space complexity, branching factor, search depth, dan jumlah iterasi.

BAB III — Perancangan dan Implementasi

Bagian ini menjelaskan bagaimana teori diterapkan menjadi program.

3.1 Arsitektur Sistem

Sistem dipisahkan menjadi antarmuka, aturan permainan, algoritma, dan eksperimen. Pembagian ini memungkinkan pengujian algoritma tanpa menjalankan Streamlit.



`game/board.py` menyediakan kelas `Board` serta fungsi `create_board()` dan `copy_board()`. `game/engine.py` menyediakan kelas `GameEngine` dan API fungsional untuk AI, UI, dan eksperimen. Antarmuka kelas dipertahankan untuk kompatibilitas, sedangkan integrasi utama menggunakan list-of-lists dan fungsi tanpa mutasi input.

MCTS menggunakan papan datar 64 elemen dan fungsi aturan internal untuk mempercepat rollout. Dengan demikian, algoritma ini belum memanggil engine bersama pada setiap transisi internalnya. Tournament tetap memvalidasi langkah dan menjalankan perubahan papan pertandingan menggunakan engine bersama. Perbedaan implementasi aturan internal ini perlu diperhatikan saat menafsirkan perbandingan kecepatan dan menjaga konsistensi.

3.2 Implementasi Game Engine

Jelaskan representasi board, validasi langkah, flip bidak, aturan pass, dan terminal state.

Dalam Proyek ini, papan direpresentasikan menggunakan list dua dimensi berukuran 8x8.

3.2.1 Representasi Board

Nilai	Keterangan
1	Bidak Hitam
-1	Bidak Putih
0	Kotak Kosong

Kondisi Awal :

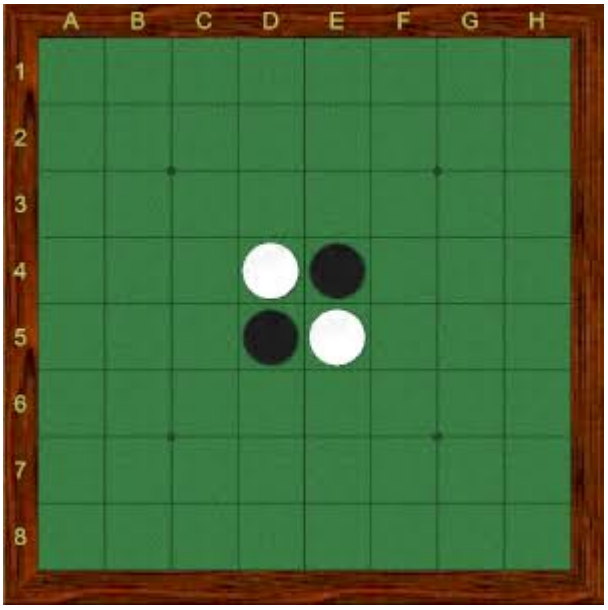
```
# posisi awal dari permainan othello

        self.grid[3][3] = self.WHITE

        self.grid[3][4] = self.BLACK

        self.grid[4][3] = self.BLACK

        self.grid[4][4] = self.WHITE
```



3.2.2 Validasi Langkah

Validasi dilakukan dengan memeriksa delapan arah dari posisi yang akan ditempati. Langkah dianggap valid apabila terdapat satu atau lebih bidak lawan yang dapat dijepit oleh bidak pemain.

a. **Arah Checking/Pemeriksaan**

```
DIRECTIONS = [
    (-1, -1), (-1, 0), (-1, 1),
    (0, -1),           (0, 1),
    (1, -1),  (1, 0),  (1, 1)
]
```

Fungsi yang berkaitan :

a. **get_captured_discs()** → Fungsi ini mencari seluruh bidak yang warna nya dapat dibalik sesuai arah checking dengan syarat bidak lawan harus dijepit oleh bidak sendiri.
b. **is_valid_moves ()** → Fungsi ini memeriksa apakah langkah pemain valid atau tidak
c. **get_valid_moves ()** → Fungsi ini mengambil semua langkah pemain yang valid

3.2.3 Flip Bidak

Setelah langkah dinyatakan valid, sistem akan membalik semua bidak lawan yang berhasil dijepit

3.2.4 Aturan Pass

Sistem harus memeriksa apakah pemain memiliki langkah legal. Jika tidak ada, giliran diberikan kepada lawan. Jika kedua pemain tidak memiliki langkah legal, permainan berakhir.

Fungsi yang berkaitan :

d. **must_pass ()** → Memeriksa apakah pemain saat ini harus pass
e. **pass_turn ()** → Melewati giliran jika langkah tidak valid
f. **get_next_player ()** → Menentukan giliran setelah langkah , termasuk pass lawan dan terminal

3.2.5 Terminal State

Permainan berakhir apabila kedua pemain tidak memiliki langkah legal atau valid
Fungsi yang berkaitan :

g. **is_terminal ()** → Permainan berakhir ketika tidak ada langkah legal tersisa
h. **get_score ()** → Menghitung jumlah bidak hitam dan putih
i. **get_winner ()** → Bidak Hitam atau putih yang memiliki jumlah terbanyak merupakan pemenang

3.3 Implementasi Alpha-Beta Pruning

Alpha-Beta ada di `algorithms/alphabeta.py` lewat `get_best_move(board, player, depth=4)`, yang mengembalikan `(move, stats)`. `player` harus 1 (Hitam)

atau -1 (Putih) dan `depth` minimal 1, kalau tidak fungsi melempar `ValueError`. Papan input tidak diubah, dan hasilnya `None` kalau pemain tidak punya langkah legal.

3.3.1 Cara kerja

1. Di root, setiap langkah legal dicoba lalu dinilai lewat pencarian rekursif.
2. Node pemain root memaksimalkan nilai (memperbarui alpha), node lawan meminimalkannya (memperbarui beta).
3. Cabang dipangkas begitu alpha lebih besar atau sama dengan beta.
4. Daun (depth 0 atau state terminal) dinilai dengan fungsi evaluasi.
5. Kalau pemain harus pass, giliran pindah ke lawan tanpa mengurangi depth.

3.3.2 Heuristic evaluation

`evaluate(board, player)` di `algorithms/evaluation.py` menilai papan dari sudut pandang `player`; nilai positif berarti menguntungkan. Pada state terminal, nilainya 0 untuk seri dan ±100.000 ditambah selisih bidak untuk menang atau kalah, sehingga kemenangan selalu mengalahkan skor heuristik. Pada state non-terminal:

skor = 25 x selisih sudut + 5 x selisih mobilitas + 1 x selisih bidak

Sudut diberi bobot terbesar karena tidak bisa dibalik lagi. Mobilitas (selisih jumlah langkah legal) lebih penting daripada jumlah bidak pada awal dan tengah permainan.

3.3.3 Batas depth

`depth` adalah sisa kedalaman, dan simpul dievaluasi saat depth mencapai 0 (bawaan 4). Kalau beberapa langkah di root bernilai sama, dipilih koordinat terkecil. Kami tidak memakai move ordering, jadi efektivitas pemangkasan bergantung pada urutan baris dan kolom.

*3.3.4 Statistik pencaria**n***

- `nodes_visited`: jumlah simpul yang diperiksa.
- `nodes_expanded`: jumlah simpul anak yang dibangkitkan, termasuk transisi pass.
- `max_depth_reached`: kedalaman terdalam (root = 0).
- `pruned_branches`: jumlah cabang yang dilewati karena pemangkasan.

3.4 Implementasi MCTS

3.4.1 Struktur Node

MCTS menggunakan struktur pohon (search tree) untuk menyimpan berbagai kemungkinan kondisi permainan. Setiap simpul atau node merepresentasikan suatu kondisi papan Othello setelah pemain melakukan langkah tertentu. Struktur node diimplementasikan

menggunakan kelas `MCTSNode`.
Setiap node memiliki beberapa atribut :

A. board → Menyimpan kondisi papan permainan.
B. player → Menyimpan pemain yang akan melakukan langkah berikutnya.
C. parent → Menunjukkan node induk.
D. children → Menyimpan daftar node anak yang telah dikembangkan.
E. visits → Menghitung berapa kali node dikunjungi.
F. total_reward → Menyimpan akumulasi reward dari simulasi.
G. move → Menyimpan langkah yang menghasilkan node tersebut.
H. mover → Menyimpan pemain yang melakukan langkah untuk mencapai node tersebut.
I. untried → Menyimpan langkah legal yang belum dikembangkan menjadi node anak.

3.4.2 Selection Menggunakan UCT

Dalam proyek ini, pemilihan node menggunakan metode Upper Confidence Bound for Trees (UCT).
UCT mempertimbangkan dua aspek:

1. Exploitation: memilih node yang memiliki rata-rata reward tinggi berdasarkan hasil simulasi sebelumnya.
2. Exploration: memberikan kesempatan kepada node yang belum banyak dikunjungi agar tidak selalu memilih langkah yang sama.

Rumus UCT yang digunakan :

$$UCT_i = \overline{X_i} + c\sqrt{\frac{\ln N}{n_i}}.$$

Cara Kerja :

1. Sistem memeriksa node yang sudah memiliki anak.
2. Setiap node anak dihitung nilai UCT-nya.
3. Node dengan nilai UCT tertinggi dipilih.
4. Proses berulang hingga menemukan node yang masih memiliki langkah legal yang belum dikembangkan.

3.4.3 Rollout Policy

Rollout merupakan tahap simulasi permainan untuk mengetahui kemungkinan hasil dari suatu kondisi papan. rollout menggunakan kebijakan random legal move, yaitu memilih langkah secara acak dari seluruh langkah legal yang tersedia. selama simulasi berlangsung, sistem tidak menggunakan strategi khusus seperti Alpha-Beta untuk menentukan langkah. Sistem hanya memilih salah satu langkah legal secara acak

Cara Kerja :

1. Sistem menyalin kondisi papan dari node yang sedang disimulasikan.
2. Sistem mencari seluruh langkah legal pemain saat ini.
3. Jika tersedia langkah legal, sistem memilih salah satu secara acak.
4. Langkah diterapkan dan bidak lawan dibalik sesuai aturan Othello.
5. Giliran berpindah ke pemain berikutnya.
6. Proses berlanjut hingga kedua pemain berturut-turut tidak memiliki langkah legal.
7. Sistem menghitung jumlah bidak untuk menentukan hasil simulasi.

Reward yang digunakan :

Hasil Simulasi	Nilai Reward
Menang	1.0
Seri	0.5
Kalah	0.0

3.4.4 Backpropagation

Backpropagation merupakan tahap memperbarui statistik node berdasarkan hasil simulasi yang telah dilakukan.

Setelah rollout selesai, sistem memperoleh hasil berupa kemenangan, kekalahan, atau seri. Hasil tersebut kemudian digunakan untuk memperbarui informasi pada node yang dilalui selama proses pencarian.

3.4.5 Pengaturan Iterations

Setiap iterasi menjalankan proses:

1. Selection.
2. Expansion.
3. Rollout.
4. Backpropagation.

Semakin banyak iterasi, semakin banyak simulasi yang dapat dilakukan oleh sistem untuk memperkirakan kualitas langkah yang tersedia.

Pada Kode kami, Jika pemain tidak memiliki langkah legal dan harus melakukan pass, fungsi pencarian dapat mengembalikan hasil tanpa melakukan simulasi. Jadi, jumlah simulasi tidak selalu sama dengan nilai iterations dalam kondisi khusus tersebut.

Setelah seluruh iterasi selesai, sistem menentukan langkah berdasarkan jumlah kunjungan node anak terbanyak. Jika jumlah kunjungannya sama, implementasi memilih koordinat dengan indeks paling kecil.

3.5 Implementasi Antarmuka

Antarmuka dibuat dengan Streamlit di `app.py` (`python -m streamlit run app.py`). Aturan permainan diambil dari `game/engine.py`, dan AI dipanggil lewat fungsi `search` di `experiments/common.py`, jalur yang sama dengan benchmark dan turnamen. Kalau modul Game Engine, Alpha-Beta, atau MCTS belum lengkap, aplikasi menampilkan pesan error impor dan tidak crash.

3.5.1 Mode dan konfigurasi

Ada tiga mode: Human vs Human, Human vs AI (pengguna memilih Hitam atau Putih), dan AI vs AI. Untuk tiap warna, pengguna memilih algoritma (`alphabeta` atau `mcts`), Depth, dan Iterations, ditambah satu Seed MCTS. Tombol New Game menerapkan pengaturan baru, sedangkan Reset mengulang permainan dengan konfigurasi yang aktif.

3.5.2 Papan dan alur permainan

Papan 8x8 berupa grid tombol dengan kolom A sampai H dan baris 1 sampai 8. Langkah legal ditandai lingkaran hijau dan hanya bisa diklik pada giliran manusia. Antarmuka juga menampilkan skor, giliran, pemberitahuan pass, dan pemenang. Giliran AI hanya jalan saat tombol Jalankan giliran AI ditekan, supaya pencarian yang mahal tidak terulang setiap Streamlit menjalankan ulang skripnya. Seed MCTS tiap langkah adalah seed dasar ditambah jumlah langkah yang sudah dimainkan, dan langkah AI dicek legal sebelum diterapkan.

3.6 Implementasi Sistem Eksperimen

Folder `experiments/` dijalankan dari baris perintah dan tidak mengimpor Streamlit sama sekali. Modulnya hanya bergantung pada `game/` dan antarmuka `get_best_move` tiap algoritma, jadi benchmark dan pertandingan otomatis bisa berjalan tanpa GUI.

- **Antarmuka bersama.** `search(algorithm, board, player, parameter, seed)` di `experiments/common.py` memvalidasi masukan, lalu memanggil `get_best_move` dengan `depth` untuk Alpha-Beta atau `iterations` dan `seed` untuk MCTS.
- **Skenario uji.** `get_scenarios()` membangkitkan tiga posisi yang pasti bisa dicapai: opening (0 langkah), midgame (24 langkah), dan endgame (50 langkah). Posisi dibentuk dengan selalu memainkan langkah legal pertama menurut urutan koordinat, jadi hasilnya selalu sama.
- **Benchmark.** `benchmark.py` mengukur satu algoritma pada satu skenario dengan 30 pengulangan (bawaan) pada salinan papan yang identik. Yang dicatat: waktu eksekusi (ms), memori puncak (`tracemalloc`, byte), dan statistik pencarian, lalu disimpan sebagai CSV. Seed MCTS tiap pengulangan adalah seed dasar ditambah nomor pengulangan.
- **Pertandingan otomatis.** `tournament.py` memainkan Alpha-Beta melawan MCTS sampai selesai. Satu pasangan terdiri dari dua game dengan warna ditukar. Seed MCTS tiap keputusan adalah seed game ditambah indeks keputusan. Pass ditangani otomatis, dan proses berhenti dengan error kalau AI memberi langkah ilegal. Per

game dicatat algoritma tiap warna, parameter, seed, pemenang, skor, jumlah langkah, dan durasi.

BAB IV — Pengujian dan Analisis

4.1 Lingkungan dan Metodologi Pengujian

Lingkungan yang digunakan pada laporan:

Komponen	**Hasil Pemeriksaan**
Prosesor	Intel Core i7-14700HX
Logical processors yang terlihat Python	28
Memori fisik yang dilaporkan OS	34.049.327.104 byte, sekitar 31,71 GiB
Sistem operasi	Windows 11, build 26200, 64-bit
Python	CPython 3.13.5, 64-bit
Streamlit	1.66.1
pandas	3.1.0
pytest	9.2.0
Pengukuran waktu benchmark	`time.perf_counter_ns()`, hasil dalam ms
Pengukuran memori benchmark	Peak allocated Python memory melalui `tracemalloc`

Konfigurasi data empiris yang tersedia adalah:

Dataset	**Algoritma dan konfigurasi**	**Posisi**	**Banyak data**	**Seed**
`results/benchmark.csv`	Alpha-Beta, depth 4	Midgame, putih mendapat giliran	30 pencarian	Tidak menggunakan pengacakan

`results/benchmark.csv`	MCTS, 500 iterasi	Midgame yang sama	30 pencarian	42–71
`results/tournament.csv`	Alpha-Beta depth 4 vs MCTS 500 iterasi	Papan awal standar	10 pairs, 20 pertandingan	Seed dasar 42; ditambah indeks keputusan MCTS

4.2 Analisis Waktu Eksekusi

Hasil dihitung per algoritma dari `results/benchmark.csv` yang telah diperbarui. Keduanya memakai papan midgame dan pemain aktif yang sama.

Statistik	Alpha-Beta depth 4	MCTS 500 iterasi
Jumlah pengulangan	30	30
Rata-rata (ms)	49,397	2.577,285
Median (ms)	48,251	2.588,316
Simpangan baku sampel (ms)	7,530	2.053,283
Minimum (ms)	41,974	505,478
Maksimum (ms)	80,163	5.436,114
Koefisien variasi (%)	15,244	79,668

Pada pengukuran ini, rata-rata waktu MCTS sekitar **52,175 kali** rata-rata Alpha-Beta. Angka tersebut adalah rasio hasil pada depth 4 dan 500 iterasi di satu posisi, bukan perbandingan

dengan anggaran komputasi setara atau bukti bahwa rasio yang sama berlaku pada semua posisi.

4.3 Analisis Penggunaan Memori

Peak allocated Python memory dihitung per pemanggilan. Berikut ringkasan dalam KiB (1 KiB = 1.024 byte):

Statistik peak memory	Alpha-Beta depth 4 (KiB)	MCTS 500 iterasi (KiB)
Rata-rata	4,003	436,467
Median	3,938	436,742
Simpangan baku sampel	0,357	1,531
Minimum	3,938	430,617
Maksimum	5,891	437,906

Rata-rata memori terlacak Alpha-Beta adalah 4.098,7 byte, sedangkan MCTS 446.942,1 byte. Pada konfigurasi ini, peak memory rata-rata MCTS sekitar 109,046 kali Alpha-Beta. Ini merupakan perbandingan alokasi yang dilacak selama fungsi berjalan, bukan rasio total penggunaan RAM aplikasi.

4.5 Analisis Efektivitas Pencarian

Statistik MCTS dari 30 pengulangan dirangkum sebagai berikut:

Metrik	Rata-rata	Rentang
`nodes_visited`	2.281,033	2.267–2.299

`nodes_expanded`	500	500–500
`max_depth_reached`	5,167	5–6
`simulations`	500	500–500
`tree_size`	501	501–501

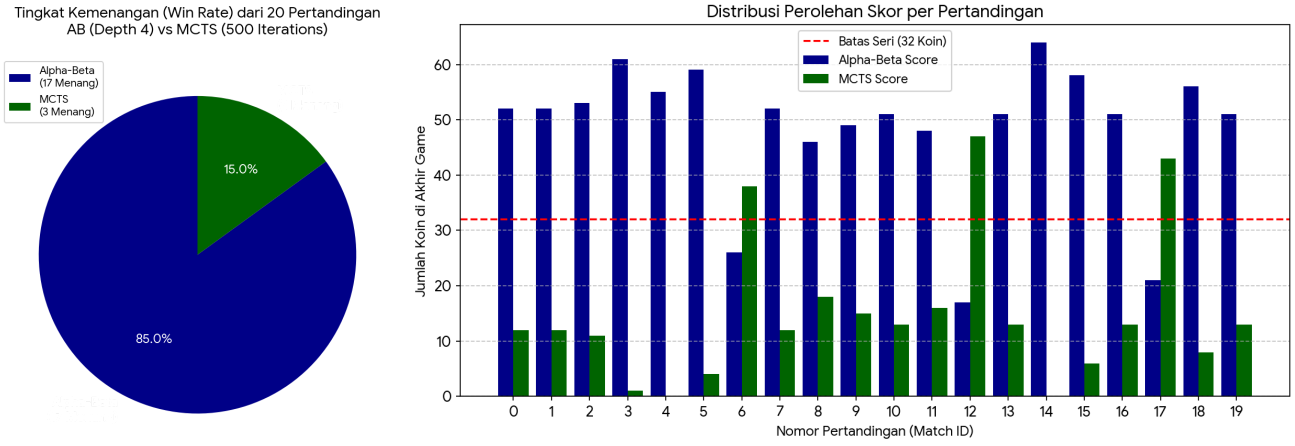
Kesamaan `simulations` dan iterations menunjukkan seluruh 500 rollout selesai pada posisi benchmark. Hubungan `tree_size = nodes_expanded + 1` juga terpenuhi karena root dihitung sebagai satu node. Pada dataset ini, setiap iterasi memang menambahkan satu node; sifat tersebut tidak harus berlaku pada seluruh posisi, misalnya jika pohon permainan yang sangat kecil sudah habis dikembangkan.

Statistik Alpha-Beta depth 4 pada 30 pengulangan adalah:

Metrik	**Nilai setiap pengulangan**
`nodes_visited`	106
`nodes_expanded`	106
`max_depth_reached`	4
`pruned_branches`	163

Kesamaan statistik sesuai dengan sifat deterministik Alpha-Beta pada papan dan urutan langkah yang sama. Nilai 163 berarti child legal yang belum dikunjungi akibat cutoff; angka tersebut bukan jumlah seluruh descendant yang berhasil dihindari. Karena benchmark Minimax tanpa pruning tidak direkam, persentase penghematan terhadap ukuran pohon Minimax penuh belum dapat dihitung. Tes unit memeriksa kesetaraan nilai dengan Minimax referensi pada posisi/depth yang sama.

4.6 Analisis Kemampuan Bermain



- Dominasi Alpha-Beta: Alpha-Beta Depth 4 menunjukkan performa jauh lebih unggul dengan win rate 85% (17 kemenangan), sedangkan MCTS 500 iterasi hanya memperoleh 15% (3 kemenangan). Hal ini membuktikan bahwa heuristic berbasis pengetahuan pakar lebih efektif dibandingkan pendekatan probabilistik MCTS pada Othello 8x8.

- Kemampuan Wipeout Alpha-Beta: Alpha-Beta berhasil mencatatkan kemenangan sempurna 55-0 pada Game 4 dan 64-0 pada Game 14. Hasil ini menunjukkan kemampuannya menguasai sudut (corner), membatasi mobilitas lawan, dan mendominasi jalannya permainan.

- Inkonsistensi MCTS: MCTS hanya berhasil menang pada Game 6, 12, dan 17. Kemenangan tersebut menunjukkan bahwa MCTS sesekali mampu mengeksploitasi kelemahan pencarian Alpha-Beta, tetapi 500 iterasi masih belum cukup untuk menghasilkan performa yang konsisten.

Kesimpulan: Pada keterbatasan sumber daya menengah hingga tinggi, Alpha-Beta Depth 4 terbukti lebih unggul dibandingkan MCTS 500 iterasi, terutama karena efektivitas heuristic dalam mengevaluasi posisi dan menentukan langkah strategis.

4.7 Analisis Berdasarkan Fase Permainan

Skenario yang dihasilkan kode diverifikasi melalui replay langkah legal. Karakteristiknya adalah:

Skenario	Penempatan setelah awal	Bidak terisi	Kotak kosong	Hitam	Putih	Pemain aktif	Langkah legal pemain aktif
Opening	0	4	60	2	2	Hitam	4
Midgame	24	28	36	8	20	Putih	5
Endgame	50	54	10	13	41	Hitam	3

Posisi ini berasal dari kebijakan pembentukan deterministik, bukan sampel acak posisi yang mewakili seluruh fase Othello. Pemilihan koordinat terkecil berulang dapat menghasilkan pola papan yang berbeda dari permainan manusia atau AI yang kuat. Oleh karena itu, satu posisi per fase harus diperlakukan sebagai kasus uji tetap.

4.8 Pembahasan Keseluruhan

Implementasi telah menghubungkan dua metode pencarian dengan aturan pertandingan, GUI, benchmark, dan tournament. Pengujian menunjukkan bahwa input papan dipertahankan oleh API pencarian, langkah yang dipilih legal pada kasus uji, seed MCTS dapat direproduksi, dan seluruh permainan dapat dijalankan sampai terminal.

Perbedaan utama kedua metode tampak pada pengendali dan penyimpanan pencarian. Alpha-Beta mengandalkan cakupan hingga depth tertentu dan kualitas heuristik. MCTS menggunakan anggaran simulasi serta mempertahankan tree untuk mengarahkan iterasi berikutnya. Banyaknya node yang diperiksa tidak mempunyai makna komputasi yang identik karena MCTS juga menjalankan rollout yang tidak dicatat sebagai node eksplisit.

BAB V — Penutup

5.1 Kesimpulan

1. **Implementasi dan integrasi.** Alpha-Beta Pruning dan MCTS berhasil diintegrasikan dengan permainan Othello, antarmuka Streamlit, serta modul eksperimen. Pengujian ulang menghasilkan 106 kasus lulus dan satu dilewati karena layar dependensi belum tersedia tidak lagi relevan.
2. **Kompleksitas dan biaya empiris.** Alpha-Beta mempunyai biaya terburuk eksponensial terhadap depth pada model branching factor tetap, sementara MCTS mengendalikan pekerjaan melalui iterasi dengan biaya selection, expansion, rollout, dan backpropagation. Pada midgame yang sama, Alpha-Beta depth 4 menghasilkan rata-rata waktu 49,397 ms dan peak memory rata-rata 4,003 KiB, sedangkan MCTS

500 iterasi menghasilkan 2.577,285 ms dan 436,467 KiB. Alpha-Beta mencatat kedua metrik lebih rendah pada konfigurasi ini, dengan catatan bahwa anggaran komputasi belum disetarakan dan kondisi runtime tidak sepenuhnya dikendalikan.

3. **Pengaruh parameter.** Depth membatasi penempatan yang ditelusuri Alpha-Beta, sedangkan iterations membatasi jumlah rollout MCTS. Dataset mengonfirmasi 500 rollout selesai dan 501 node tree pada konfigurasi yang diuji. Pengaruh peningkatan parameter terhadap kurva waktu, memori, dan kekuatan bermain belum terukur karena belum ada penyapuan parameter.
4. **Kemampuan bermain dan fase.** Setelah 20 Pertandingan, hasil turnamen menunjukkan dominasi mutlak Alpha-Beta (Depth 4) dengan Win Rate 85% (17 kemenangan) atas MCTS (500 Iterations). Saat Alpha-Beta menang, ia tidak hanya unggul tipis, melainkan mampu mengeksekusi strategi secara brutal hingga mencapai kondisi wipeout (menang telak tanpa menyisakan satu pun koin lawan, contoh: skor 55-0 pada Game 4 dan 64-0 pada Game 14) berkat evaluasi heuristik statisnya yang sangat mematikan di kedalaman 4.

Daftar Pustaka

1. Knuth, D. E., dan Moore, R. W. (1975). *An Analysis of Alpha-Beta Pruning*. Artificial Intelligence, 6(4), 293–326. Artikel penerbit.
2. Kocsis, L., dan Szepesvári, C. (2006). *Bandit Based Monte-Carlo Planning*. ECML 2006, 282–293. Naskah publik.
3. Browne, C. B., dkk. (2012). *A Survey of Monte Carlo Tree Search Methods*. IEEE Transactions on Computational Intelligence and AI in Games, 4(1), 1–43. Repositori University of Essex.
4. World Othello Federation. *Official Rules for the Game Othello*. Aturan resmi.
5. Python Software Foundation. *tracemalloc — Trace Memory Allocations*, dokumentasi Python 3.13. Dokumentasi.
6. Python Software Foundation. *time.perf_counter_ns*, dokumentasi Python 3.13. Dokumentasi.